

WINDOWS RDP BRUTE-FORCE

DETECTION LAB

A Splunk SIEM Detection Engineering Project

Prepared by

Vijay S

Cybersecurity Student — SOC Analyst & Penetration Tester Track

June 29, 2026

Executive Summary

This project documents the design, build, and validation of a Windows RDP brute-force detection capability using Splunk Enterprise as the SIEM platform. The lab simulates a realistic attack scenario — a remote desktop credential brute-force attempt against a Windows 11 host — and builds a complete detection and alerting pipeline around it, from raw Windows Security Event Log collection through to a scheduled, throttled Splunk alert and an analyst-facing dashboard.

The project was deliberately scoped to fill a gap in a prior all-Linux SOC lab portfolio piece, demonstrating production-style Windows log forwarding, Windows Event ID analysis (4624/4625), SPL detection engineering, and real-world infrastructure troubleshooting — including a cross-timezone clock synchronization issue that was diagnosed and resolved mid-project.

Lab Objective

Build an end-to-end detection pipeline capable of identifying RDP brute-force activity against a Windows host, using only native Windows Security Event Logs (no third-party EDR), and surface that detection through an automated Splunk alert and dashboard suitable for SOC analyst triage.

Specific goals:

- Stand up a Windows 11 Enterprise VM as an isolated attack target, forwarding logs to Splunk Enterprise via the Splunk Universal Forwarder.
- Execute a realistic RDP brute-force attack using Hydra from a Kali Linux attacker VM.
- Engineer an SPL detection search that correctly identifies repeated failed logon attempts (Event ID 4625) by account and source address.
- Configure a scheduled, throttled Splunk alert that fires automatically on qualifying activity without flooding the analyst with duplicate notifications.
- Build a two-panel analyst dashboard summarizing the detection and visualizing the attack timeline.

Lab Architecture & Environment

All systems run as VMware Workstation virtual machines on a single isolated host-only network (VMnet1), with no internet egress — ensuring the attack traffic and log data never leave the lab environment.

Attacker	Kali Linux — 192.168.100.129
Target	Windows 11 Enterprise — 192.168.100.130 (hostname DESKTOP-PO86RHM)
Test Account	victim (added to Remote Desktop Users)
SIEM Host	Splunk Enterprise 10.4.0 — 192.168.100.1 (VMnet1 host adapter), receiving on port 9997
Log Forwarding	Splunk Universal Forwarder 10.4.0, forwarding Windows Security and System event logs
Network	VMware VMnet1 (host-only, isolated, no internet access)
Attack Tool	Hydra v9.6 (RDP module)

Lab Setup & Infrastructure Build

Before any attack or detection work could begin, the Windows 11 target had to be built from scratch and wired into the existing Splunk/Kali lab. This phase surfaced several infrastructure-level problems typical of virtualized lab environments, each of which required independent diagnosis rather than a documented one-step fix.

Virtual Machine Creation

A new Windows 11 Enterprise VM was created in VMware Workstation (4 GB RAM, 2 vCPUs, 60 GB disk), attached to the VMnet1 host-only adapter to keep it on the same isolated network as the existing Kali Linux and Metasploitable2 VMs.

Issue 1 — Vulkan Renderer Crash

Powering on the new VM immediately produced an unrecoverable VMware error referencing `bora\mks\backends\vulkan\vrSwapchain.c`. This is a known defect in VMware Workstation's newer Vulkan-based 3D rendering backend, triggered by a mismatch with certain host GPU drivers. Resolution: VM Settings → Display → uncheck “Accelerate 3D graphics,” forcing the VM back onto the older, stable 2D renderer. This has no impact on the lab's requirements, since none of the work (Windows install, Splunk forwarding, RDP brute-forcing) depends on GPU acceleration.

Issue 2 — Windows Activation on an Isolated Network

Windows 11 Enterprise required online activation, which is impossible on the air-gapped VMnet1 network by design. The VM's network adapter was temporarily switched to NAT (VMnet8) to reach Microsoft's activation servers, activated with the `slmgr /ato` command, and then switched back to VMnet1 once activation completed — restoring full network isolation for the remainder of the lab.

This step itself uncovered a secondary fault: VMware's NAT service failed because the VMnet8 virtual network had never been configured on this host. It was created via VMware's Virtual Network Editor (Edit → Virtual Network Editor → Add Network → VMnet8, NAT type) before the activation workaround could proceed.

Issue 3 — RDP Enablement and Verification

Remote Desktop was enabled on the Windows 11 VM (Settings → System → Remote Desktop), and a dedicated low-privilege test account (victim) was created and added to the Remote Desktop Users group — deliberately separate from the administrative account, so the brute-force target mirrors a realistic standard-user attack surface. Port 3389 reachability was independently confirmed from the Kali VM using an Nmap scan against 192.168.100.130 before any attack tooling was run, isolating “is the service reachable” from “does the attack work” as two separate checks.

Issue 4 — Splunk Universal Forwarder Configuration

The Splunk Universal Forwarder 10.4.0 was installed on the Windows 11 VM and configured to collect Security and System event logs. Writing its `outputs.conf` file (which tells the forwarder where to send data) hit two dead ends before succeeding: File Explorer's default text editor refused to open the file due to a broken file-type association, and the modern Windows 11 Notepad app failed to launch under “Run as administrator” (a known limitation of Store-packaged apps). Both were bypassed by writing the file directly from an elevated PowerShell session:

```
Set-Content -Path "C:\Program Files\SplunkUniversalForwarder\etc\system\local\
outputs.conf" -Value "[tcpout]\ndefaultGroup =
my_indexers\n[n[tcpout:my_indexers]\nserver = 192.168.100.1:9997"
```

Verified afterward with Get-Content, producing the expected four-line configuration:

```
[tcpout]
defaultGroup = my_indexers

[tcpout:my_indexers]
server = 192.168.100.1:9997
```

Issue 5 — Missing Host Adapter and Blocked Forwarder Traffic

Even with outputs.conf correctly written, the host's own VMnet1 adapter was found to be missing entirely — the “Connect a host virtual adapter” option had never been enabled in Virtual Network Editor. Enabling it gave the host machine itself a 192.168.100.1 address on the lab network, which Splunk Enterprise needed in order to listen for forwarder connections from that subnet.

With the host adapter in place, the forwarder still showed zero incoming events. A direct connectivity test from the Windows VM isolated the cause precisely:

```
Test-NetConnection -ComputerName 192.168.100.1 -Port 9997
```

returned TcpTestSucceeded: False. Enabling a receiving port in Splunk's web UI only opens it at the application layer; the host's own Windows Defender Firewall still had no inbound allow rule for that port from an external subnet. A new inbound rule (TCP 9997, all profiles, Allow) was added via Windows Defender Firewall with Advanced Security, after which the identical test returned TcpTestSucceeded: True, and forwarder data began arriving in Splunk Enterprise.

Taken together, these five issues — none of which had an obvious single cause from the symptom alone — represent the bulk of the actual engineering effort in this project's setup phase, and were resolved through the same evidence-first method used later for the clock synchronization fault: reproduce, isolate variables with a minimal test, fix the narrowest cause, and re-verify.

Attack Execution

With the forwarding pipeline confirmed operational, a brute-force credential attack was launched from the Kali Linux VM against the Windows 11 target's RDP service (port 3389), using a small custom wordlist containing several incorrect passwords followed by the account's actual password.

Command used:

```
hydra -l victim -P /tmp/wordlist.txt rdp://192.168.100.130
```

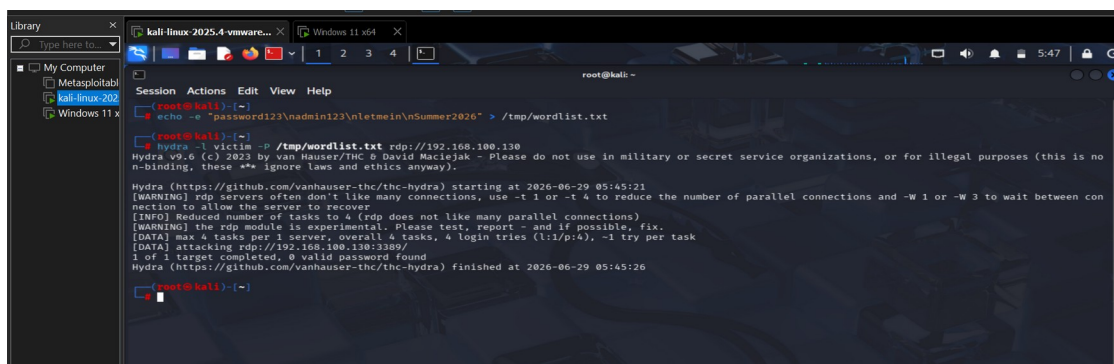


Figure 1 — Hydra RDP brute-force attempt executed from Kali Linux against the victim account.

A known quirk of Hydra's experimental RDP module is that it frequently reports “0 valid password found” even when a correct password was supplied. A later test run (see Figure 9) confirmed this directly: Hydra logged the correct credential pair (victim / Summer2026) while still reporting zero valid passwords — the underlying login attempts succeeded at the protocol level even though the module's own success-detection logic did not recognize it. This did not affect the validity of the attack as a detection data source, since Windows itself independently logs every authentication attempt regardless of what the attacking tool believes happened.

Detection Engineering

Raw Event Analysis

Each failed RDP logon generates a Windows Security Event ID 4625. Inspecting the raw events in Splunk surfaced an important field-extraction quirk: the `Account_Name` field is multivalued for these events, containing two separate values within a single event — a blank Subject account name (“-”) and the real Target account name (the account actually being attacked).

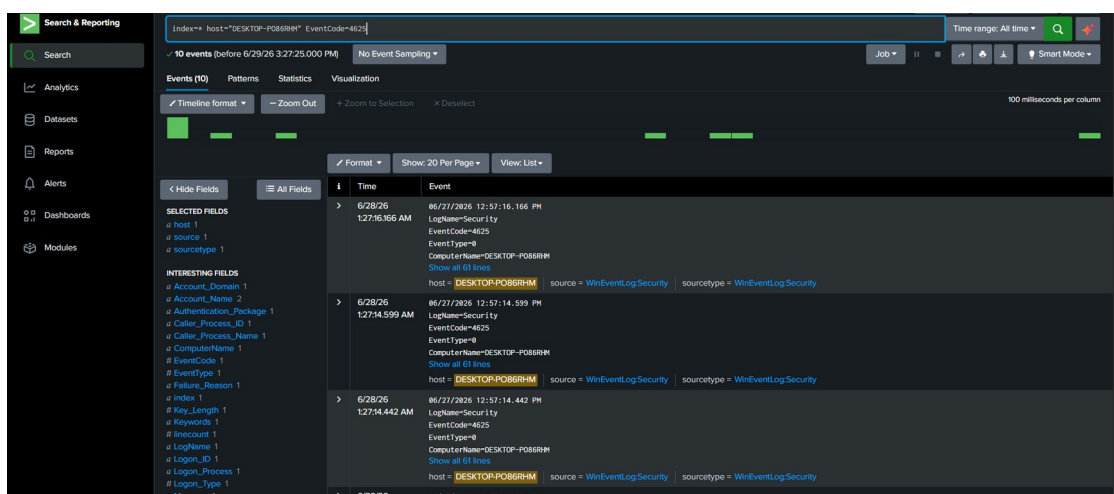


Figure 2 — Raw 4625 events. The `Account_Name` field carries two values per event, requiring explicit handling.

Building the Search

The base detection search aggregates failed logon events by account and source address, flagging any pairing that crosses a minimum attempt threshold:

```
index=* host="DESKTOP-P086RHM" EventCode=4625
| eval Account_Name=mfilter(Account_Name!="")
| stats count by Account_Name, Source_Network_Address
| where count >= 3
```

Two lessons emerged while building this search. First, filtering with a bare `Account_Name!=""` condition directly on the raw search excludes every 4625 event entirely — because that condition is evaluated against a multivalue field where one of the two values is always `"`, Splunk treats the event as failing the filter outright. The correct approach is to first collapse the multivalue field with `mfilter()`, then proceed. Second, the native Windows field for the source IP address in Security event logs is `Source_Network_Address`, not the Splunk Common Information Model field `src_ip` — a naming mismatch that initially produced silent empty results.

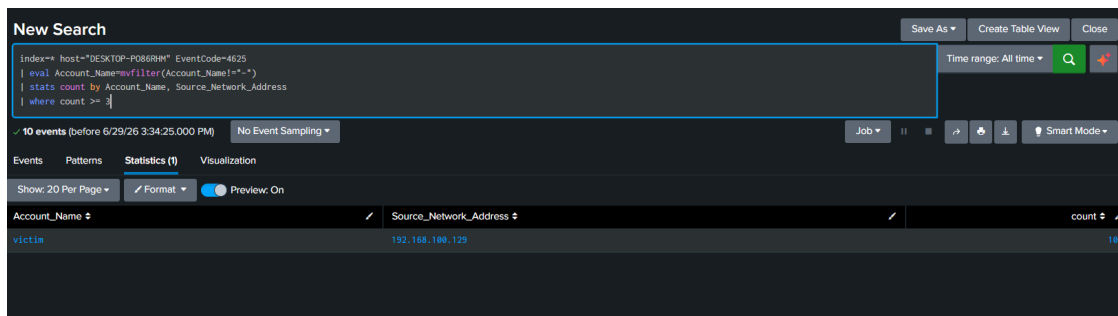


Figure 3 — Corrected detection search isolating the victim account and attacker source address, well above the alert threshold.

Alert Configuration

The detection search was saved as a scheduled Splunk alert, configured to run continuously every five minutes against a rolling 15-minute lookback window — a 3x overlap that prevents events from being missed at window boundaries.

Alert Name	Windows RDP Brute Force Alert
Time Range	Last 15 minutes
Cron Schedule	* / 5 * * * * (every 5 minutes)
Trigger Condition	Number of Results is greater than 0
Trigger Mode	For each result
Throttle	Enabled — suppress for 3,600 seconds (1 hour)
Trigger Action	Add to Triggered Alerts
Severity	Medium
Expires	24 hours

Throttling was a deliberate design choice: without it, an ongoing brute-force attempt would generate a fresh alert entry every five minutes for the same attacker for as long as the attack continued, flooding the Triggered Alerts queue with duplicates. A one-hour suppression window ensures each distinct attacker is surfaced once, then re-surfaced only if the activity is still ongoing an hour later — balancing analyst signal against alert fatigue.

Figure 4 — Final alert configuration: 5-minute cron schedule with a 1-hour throttle window.

Figure 5 — Alert saved successfully and enabled.

Troubleshooting: Clock Synchronization Issue

After saving the alert, an initial validation pass showed zero triggered events despite a confirmed brute-force attack having just been run. A manual re-run of the detection search scoped to “Last 15 minutes” also returned zero results — even though the same search scoped to “All time” correctly found the attack data. This pattern

(data exists, but only relative time windows fail) is a classic symptom of a clock or timezone mismatch between the data source and the search head, rather than a logic error in the search itself.

Root cause analysis traced the discrepancy to the Windows 11 VM's system clock: the VM's timezone was set to a U.S. zone while the Splunk Enterprise host (and the analyst's actual location) operates on India Standard Time — a roughly 10–13 hour offset depending on daylight adjustments. Compounding the issue, Windows' built-in automatic time synchronization (which relies on reaching time.windows.com over the internet) was silently failing, because the Windows VM sits on the isolated, internet-less VMnet1 host-only network by design.

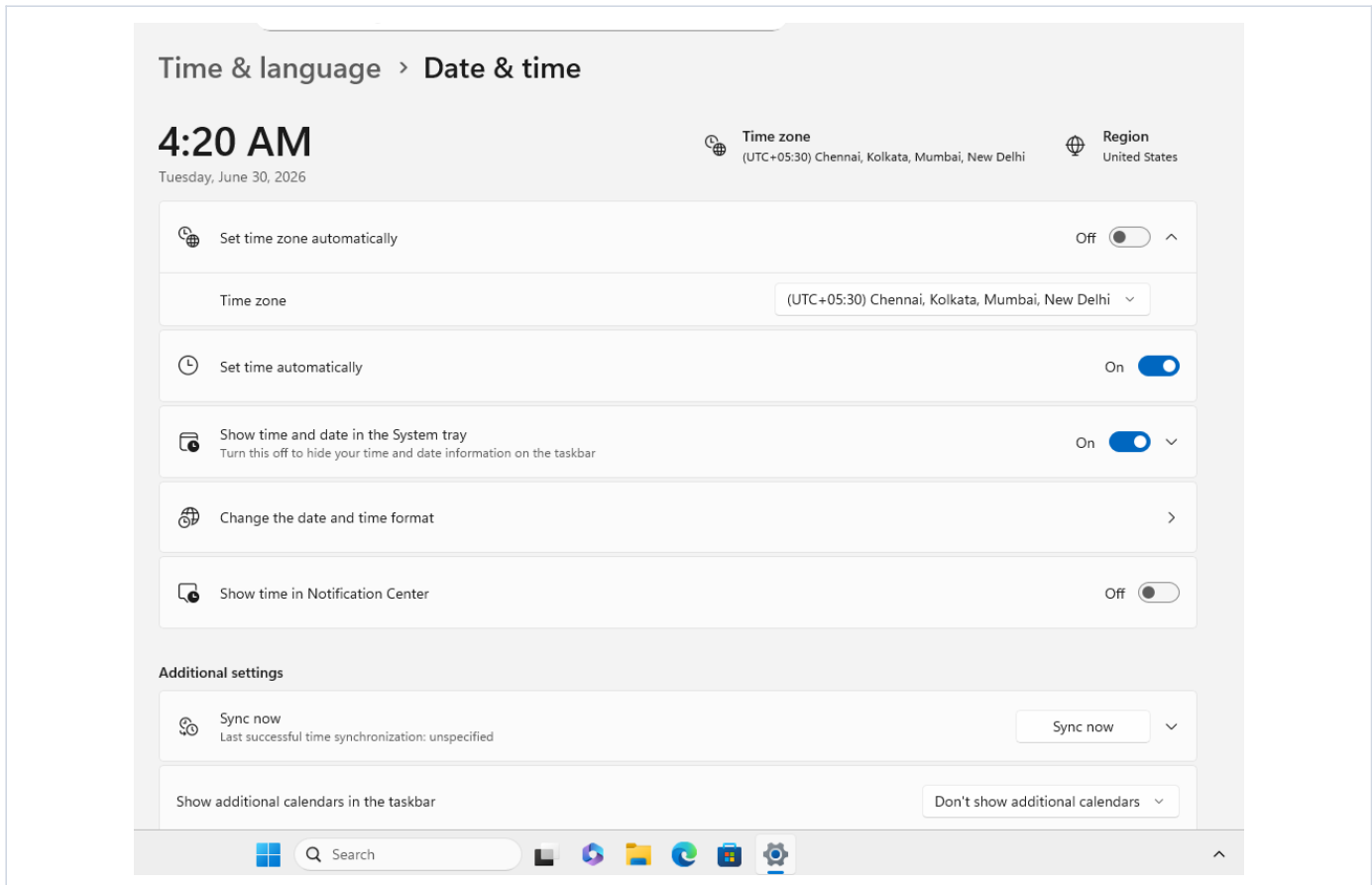


Figure 6 — VM clock before remediation: wrong timezone, and “Last successful time synchronization: unspecified” confirms the internet-based NTP sync was never working.

Remediation was applied in three steps:

- Set the VM's timezone explicitly to (UTC+05:30) Chennai, Kolkata, Mumbai, New Delhi to match the Splunk host.
- Enabled “Synchronize guest time with host” in VMware Workstation's VM Settings → Options → VMware Tools — this pulls accurate time directly from the host machine over the VM's internal connection, requiring no internet access.
- Restarted the Splunk Universal Forwarder service on the Windows VM to ensure it picked up the corrected system clock rather than any cached time offset.

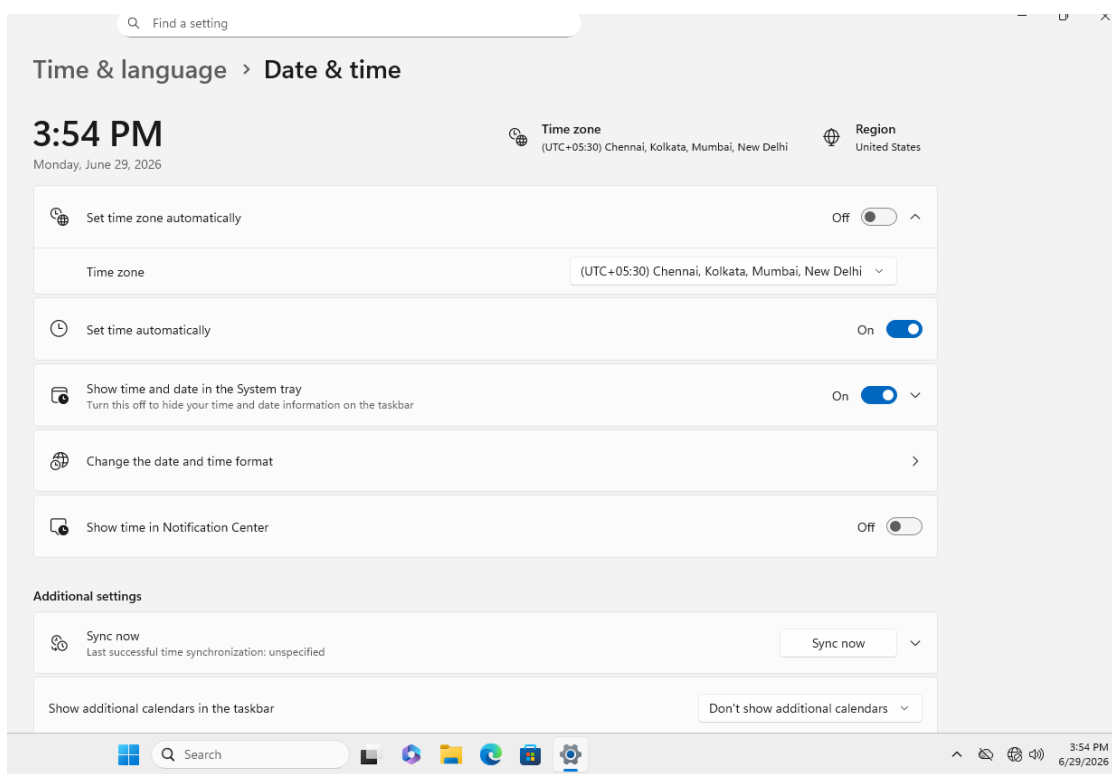


Figure 7 — VM clock after remediation: correct date, time, and timezone, matching the host.

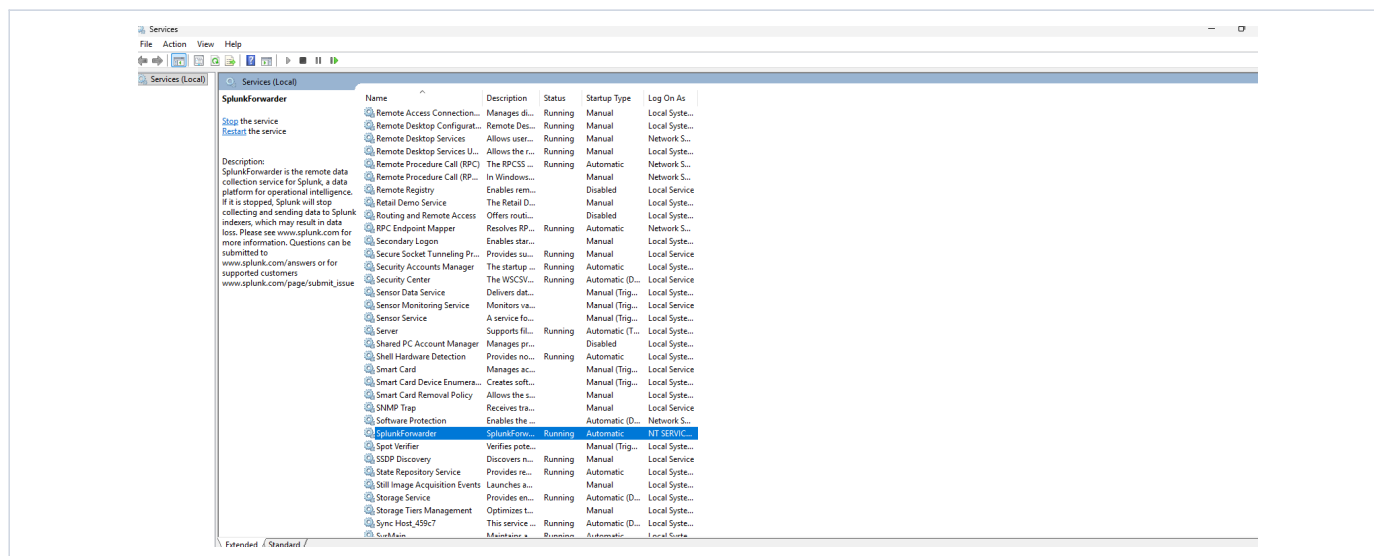


Figure 8 — Splunk Universal Forwarder service confirmed Running after restart.

This troubleshooting sequence is included in this report deliberately: time synchronization issues across virtualized, air-gapped lab environments are a realistic and common SOC operational problem, and the diagnostic method used here — isolating whether the issue is data-existence versus time-window scoping by comparing an “All time” search against a relative-time search — is directly transferable to production SIEM troubleshooting.

Alert Verification

With the clock corrected, a fresh Hydra attack was executed and the detection search was re-run scoped to “Last 15 minutes” — the same window the live alert uses — to confirm the fix held under realistic conditions.

```

root@kali: [~]
# hydra -l victim -P /tmp/wordlist.txt rdp://192.168.100.130
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-29 06:34:51
[WARNING] rdp servers often don't like many connections, use -t 1 or -t 4 to reduce the number of parallel connections and -W 1 or -W 3 to wait between con
nection to allow the server to recover
[INFO] Reduced number of tasks to 4 (rdp does not like many parallel connections)
[WARNING] the rdp module is experimental. Please test, report - and if possible, fix.
[DATA] max 4 tasks per 1 server, overall 4 tasks, 4 login tries (l:1/p:4), -1 try per task
[DATA] attacking rdp://192.168.100.130:3389/
[3389][rdp] account on 192.168.100.130 might be valid but account not active for remote desktop: login: victim password: Summer2026, continuing attacking t
he account.
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-29 06:34:52
root@kali: [~]

```

Figure 9 — Follow-up Hydra run. Note the log line confirming the correct password (Summer2026) was attempted, despite Hydra's own “0 valid password found” summary.

The screenshot shows the Splunk Enterprise search interface. The search bar contains the query: `index=* host="DESKTOP-P0689R4" EventCode=4625 | eval Account_Name=filter(Account_Name!="") | stats count by Account_Name, Source_Network_Address | where count > 3`. The time range is set to "Last 15 minutes". The search results show 4 events. The first event is highlighted, showing `Account_Name: victim` and `Source_Network_Address: 192.168.100.129` with a count of 4.

Figure 10 — Detection search correctly returns results within a real “Last 15 minutes” window, confirming the clock fix resolved the issue.

Shortly afterward, the scheduled alert fired automatically on its next 5-minute cron tick — with no manual search required — confirming the full pipeline (attack → Windows Event Log → Universal Forwarder → Splunk index → scheduled SPL search → throttled alert) was working end-to-end exactly as designed.

The screenshot shows the configuration page for the "Windows RDP Brute Force Alert". The alert is enabled, with the app set to "search", permissions set to "Private", and owned by "jacob". The trigger condition is "Number of Results is > 0". The trigger history shows one event triggered at 2026-06-29 16:05:02 India Standard Time.

Figure 11 — Alert Trigger History showing the alert fired automatically at 16:05:02 India Standard Time, on live attack data.

Dashboard

A two-panel analyst dashboard was built to summarize the detection at a glance. The first panel presents the detection search results as a table (account, source address, attempt count); the second is a column chart plotting failed (4625) versus successful (4624) logons over time for the victim account, visually narrating the breach — a cluster of failed attempts immediately followed by a successful logon.

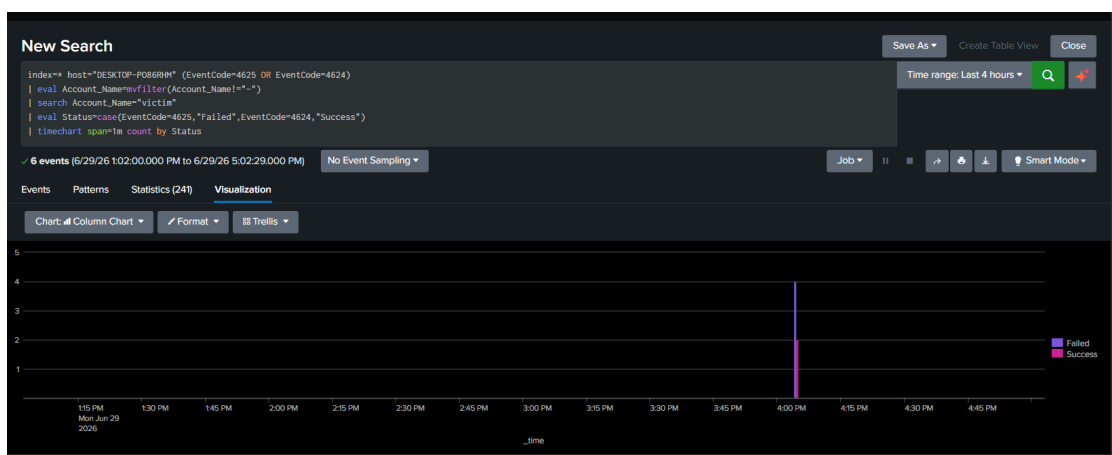


Figure 12 — Failed vs. successful logon timeline, scoped to a 4-hour window for readable bar resolution.

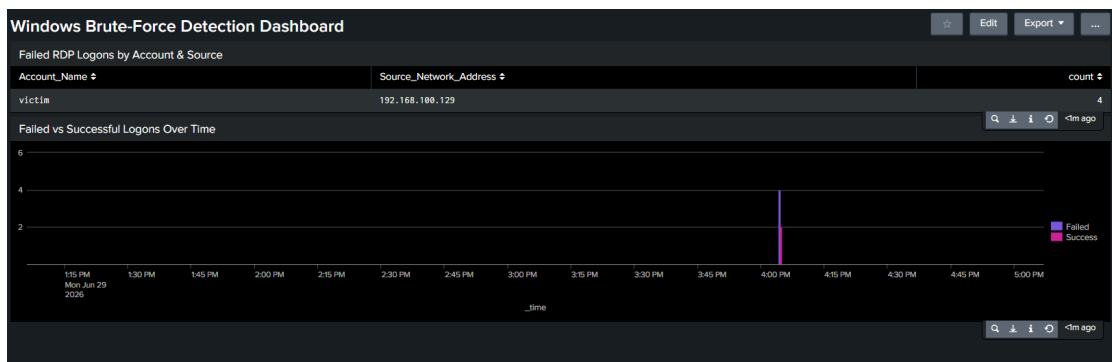


Figure 13 — Completed two-panel “Windows Brute-Force Detection Dashboard.”

Skills Demonstrated

- Windows Security Event Log analysis (Event IDs 4624 / 4625) and Windows authentication forensics.
- Splunk Universal Forwarder deployment and Windows-to-Splunk log forwarding configuration.
- SPL detection engineering, including multivalue field handling with `mvfilter()` and CIM-vs-native field name reconciliation.
- Splunk alert engineering: scheduling, trigger conditions, and throttle/suppression design to manage alert fatigue.
- Splunk dashboard design for analyst-facing detection summaries and attack visualization.
- Adversary simulation using Hydra against the RDP protocol, including working around known tool-reporting quirks.

- Systematic infrastructure troubleshooting: diagnosing and resolving a cross-timezone VM clock synchronization fault in an air-gapped lab network.
- VMware Workstation virtual lab administration: isolated host-only networking, VM Tools time synchronization, and Windows service management.
- Hypervisor-level troubleshooting: resolving a Vulkan 3D-rendering crash, configuring a missing NAT virtual network, and enabling a missing host-only virtual adapter.
- Windows network diagnostics using Test-NetConnection to isolate application-layer versus OS-firewall-layer connectivity failures.
- Operating system licensing/activation management for isolated, air-gapped virtual machines.

Conclusion

This lab delivers a complete, validated detection pipeline for RDP brute-force activity on Windows, built and debugged from first principles rather than from a pre-packaged detection content pack. Beyond the detection logic itself, the project surfaced and resolved two realistic engineering obstacles — a Windows multivalue field-extraction quirk and a cross-timezone clock synchronization fault — both of which required the kind of systematic, evidence-based troubleshooting expected of a working SOC analyst.

Combined with the earlier Linux-focused Metasploitable2 SOC lab, this project closes the Windows-detection gap in the broader portfolio and demonstrates production-style log forwarding rather than manual log upload, reinforcing a consistent end-to-end methodology: lab build → attack execution → detection engineering → alerting → visualization → documentation.